Ethics, Law & Responsible Data Collection

Week 2 · Foundations module · Textbook Chapter 2

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

August 24, 26 & 28, 2026

This week at a glance

This week covers the rules that govern collection: what is legal, what a site permits, and what is ethical.

Monday | Concepts & notebook intro

- The legal landscape, `robots.txt`, and a five-question ethical framework

Wednesday | Notebook lab

- Get `ch-02-ethics.ipynb` from GitHub and launch it in Jupyter
- Then: `robots.txt`, User-Agent headers, rate limiting — and building `responsible_get()`

Friday | Textbook revisions

- GitHub, hands-on — and everyone files their first issue against Chapter 2

Companion notebook

`ch-02-ethics.ipynb`

Bring a laptop

Wednesday is hands-on — we send live requests and watch what the server sees.

1. Monday · Concepts

The ethics of retrieval

"Web scraping" is a slightly pejorative phrase. Data is **valuable**. Other people spent time, money, and effort to create and host what you are about to retrieve for free.

Can I scrape this?

Should I scrape this?

How do I responsibly scrape this?

We need a decision framework from **law**, **skills**, and **ethics**.



With great power comes great responsibility.

Reminder

We will discuss the law but **nothing in this or any other lecture is legal advice**. Talk to a lawyer before web scraping a private website that might make rich people mad.

The Computer Fraud and Abuse Act (1986)

The **Computer Fraud and Abuse Act** was enacted in 1986 during a national panic about "computer crime."

> *"Whoever intentionally accesses a computer without authorization or exceeds authorized access, and thereby obtains information from any protected computer… and as a result of such conduct, recklessly causes damage or loss"*

The CFAA is a **criminal** statute. That makes the question "was this access authorized?" the difference between a research method and a federal charge.

Every data collection decision we make in this class is subject to a criminal statute written before a decade before the the web.



Aaron Swartz

Aaron Swartz was an influential hacker from the 2000s: RSS, Creative Commons, web.py, Markdown, Reddit

In 2010 and 2011, Swartz used MIT's open network to bulk download academic articles from JSTOR *maybe* with the intent to break the paywall: Guerille Open Access Manifesto

Federal prosecutors charged him in 2011 under the CFAA's "unauthorized access" rule carrying several decades of jail time

He died by suicide in 2013, at 26.

"The Internet's Own Boy: The Story of Aaron Swartz" (2014)



Consequences

Federal prosecutors have substantial discretion over how aggressively to prosecute charges. After Swartz, researchers and hacktivists became much more worried about CFAA prosecutions.

Cambridge Analytica

Cambridge Analytica was a political consulting firm active in the 2016 elections in both US and UK.

A researcher collected data on millions of Facebook users through the platform's API.

Users completed a survey about political identity but Facebook's Graph API allowed the researcher to pull their entire friend list

"The Great Hack" (2019, Netflix)



Consequences

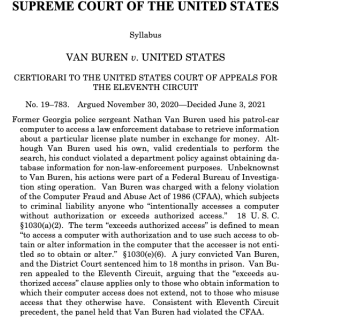
Platforms shifted their stance from permissive access and began to see researchers as liabilities and shut down access maybe to protect users' privacy but mostly to avoid bad press.

Van Buren v. United States (2021)

A police sergeant ran a license plate search in a database for a bribe. He was *authorized* to use this database. Prosecutors charged him under the CFAA's "exceeds authorized access"

The Supreme Court ruled adopted a "gates-up-or-down" reading: the CFAA targets entering areas you are not entitled to enter but does *not* reach the misuse of information you were allowed to see.

After *Van Buren*, is is more difficult for prosecutors to argue that breaking a website's terms of service rises to a federal crime. But the CFAA is not the *only* federal law for prosecuting cybercrime.



SUPREME COURT OF THE UNITED STATES

Syllabus

VAN BUREN *v.* UNITED STATES

CERTIORARI TO THE UNITED STATES COURT OF APPEALS FOR THE ELEVENTH CIRCUIT

No. 19–783. Argued November 30, 2020—Decided June 3, 2021

Former Georgia police sergeant Nathan Van Buren used his patrol-car computer to access a law enforcement database to retrieve information about a particular license plate number in exchange for money. Although Van Buren used his own, valid credentials to perform the search, his conduct violated a department policy against obtaining database information for non-law-enforcement purposes. Unbeknownst to Van Buren, his actions were part of a Federal Bureau of Investigation sting operation. Van Buren was charged with a felony violation of the Computer Fraud and Abuse Act of 1986 (CFAA), which subjects to criminal liability anyone who "intentionally accesses a computer without authorization or exceeds authorized access." 18 U. S. C. §1030(a)(2). The term "exceeds authorized access" is defined to mean "to access a computer with authorization and to use such access to obtain or alter information in the computer that the accesser is not entitled so to obtain or alter." §1030(e)(6). A jury convicted Van Buren, and the District Court sentenced him to 18 months in prison. Van Buren appealed to the Eleventh Circuit, arguing that the "exceeds authorized access" clause applies only to those who obtain information to which their computer access does not extend, not to those who misuse access that they otherwise have. Consistent with Eleventh Circuit precedent, the panel held that Van Buren had violated the CFAA.

Consequences

CFAA only applies to barriers in code not term-of-use violations, removing a tool prosecutors and platforms had been able to use against researchers and activists.

hiQ Labs v. LinkedIn (2017–2022)

hiQ is an analytics company that scraped **public** LinkedIn profiles. LinkedIn sent a cease-and-desist and blocked it in 2017.

Based on *Van Buren* (2021), the Ninth Circuit held that scraping publicly accessible data does **not** violate the CFAA. hiQ never bypassed a login page or agreed to terms of service to view the data itself.

hiQ **lost** on LinkedIn's **breach-of-contract** claim after it continued to scrape despite agreeing to LinkedIn's User Agreement and used fake accounts to get around LinkedIn's rate limiting.



Consequences

Van Buren limited the ability to use CFAA to prosecute public web scraping, but hiQ still lost under contract law. Scraping *public* data got legally safer; ignoring the Terms of Service to scrape did not.

Sandvig v. Barr (2020)

Researchers wanted to run **audit studies** of algorithmic discrimination with fake profiles. This method violates the Terms of Service of those sites. The researchers feared prosecution under the CFAA.

Violating a website's Terms of Service is **not** a CFAA crime. Enforcing TOS would turn each website into its own criminal jurisdiction.

Audit studies are well-established method offline and are the main source of evidence about algorithmic discrimination.



Consequences

Platforms still write their own Terms of Service and can lead to civil suits for breach of contract, even after *Sandvig* removed the threat of federal prosecution.

Meta v. Bright Data — Meta's breach-of-contract claims failed. Bright Data held Facebook/Instagram accounts but the scraping happened while *logged out*.

X Corp. v. Bright Data — dismissed on different grounds: X's users, not X, hold rights in their posts; X could not use its ToS to claim ownership over content it does not own.

These are trial-court rulings, but the direction is consistent with *hiQ*: platforms are struggling to use contract law to fence off public data.



Consequences

A contract needs terms that clearly reach the conduct at issue and a company cannot use its terms to claim ownership over content or data it does not itself own.

The New York Times v. OpenAI

The New York Times sued OpenAI and Microsoft to block its infringing collection of copyrighted content to train a statistical model. Or is it fair use?

Publishers hardened their sites against crawlers: within a year of the suit, most major news sites blocked OpenAI's GPTBot

AI-specific directives appeared in `robots.txt`, distinguishing crawlers for **training** from crawlers for **search**

Exclusive licensing deals carved up access, e.g. the AP, *Financial Times*, and News Corp. struck paid deals with OpenAI while others sued


The New York Times

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.

Share full article 1.3K

Consequences

A site that blocks OpenAI also blocks your class project. Robots.txt cannot tell a researcher's crawler from a commercial one. Ch. 3 covers this as **enclosure**.

State privacy law

The U.S. has no comprehensive federal privacy law, so states filled the gap. California wrote the **CCPA/CPRA** first. By 2026 close to twenty states have comprehensive statutes. They include the **Colorado Privacy Act**, which covers data about Colorado residents.



They import GDPR-like concepts: **personal data rights**, **purpose limitation**, and obligations on whoever processes the data.

The **GDPR** itself reaches EU residents' personal data regardless of where *you* are sitting.

If your scraped dataset contains personal information about residents of these places, these laws can apply to **you**.

Consequences

Collecting data about individuals still means assuming some personal risk in an increasing number of jurisdictions. Collect the minimum that answers your question.

Terms of Service as quasi-law

Although CFAA exposure has narrowed, the Terms of Service still constrains what you can legally scrape from most websites

The agreement is also **asymmetric**: You accepted it once by clicking and the platform can amends it whenever it wants to say almost anything.

Every major platform's ToS restricts automated collection. When you created an account, you agreed. Consequences run from account suspension to IP blocking to litigation.



Consequences

We will run code that may violate CFAA and platforms' ToS in educational or research capacities. That insulates us from some liability but those protections stop with commercial or abusive applications.

Reading a ToS or privacy policy

These documents are long, and few people read them from start to finish. Search them instead.

Keywords worth a Ctrl-F:

`access` | `script` | `scrape` | `crawl` | `automated` | `bot` | `API` | `research` | `data` | `abuse` | `rate` | `machine learning`

Four questions to come out with:

1. Is **automated access** addressed at all?
2. Is there a **research or academic** exception?
3. What does it say about **personal data** and **AI training**?
4. When was it **last updated** — and what changed?



Activity: audit a platform you use

Select a platform that you use. Scroll to the bottom and open its Terms of Service or Privacy Policy.

1. Search the document for the keywords.
2. Answer the four questions.
3. Find one clause that surprised you.

Does this platform permit the data collection that you plan for this class? Second, how sure are you?

Report back in pairs. We will collect the surprising clauses on the board.

Four questions

1. Is **automated access** addressed at all?
2. Is there a **research or academic** exception?
3. What does it say about **personal data** and **AI training**?
4. When was it **last updated** — and what changed?

Discussion: four articles on AI scraping

Each group takes one article:

1. **The terms move** — Tan (2024), on ToS rewritten to permit AI training
2. **The web pushes back** — Koebler (2024a), on measurable mass blocking
3. **The defense misfires** — Koebler (2024b), on sites blocking the wrong bots
4. **The rules get ignored** — Mehrotra et al. (2024), on scraping around a block

Questions for each group

1. What **changed**, concretely?
2. Who **gains** and who is **harmed**?
3. What could it mean for what **we** do in class?

The four articles report four related changes:

1. **Terms change.** Platforms amend them to permit AI training on data that users already shared. In February 2024 the FTC warned that doing this retroactively may be "unfair or deceptive" (Tan 2024).
2. **Sites add restrictions.** Of 14,000 sites studied, full AI restrictions went from about 1% to 5–7% in one year. **28%** of the most actively maintained sources added restrictions (Koebler 2024a).
3. **The blocks often fail.** Reuters and Condé Nast blocked two *retired* Anthropic crawlers but not the active one. iFixit received about 1 million requests in one day. Read the Docs spent $5,000 on bandwidth (Koebler 2024b).
4. **Some operators ignore the rules.** Perplexity reached blocked pages from an unpublished IP address. It then fabricated a claim that a named police officer had committed a crime (Mehrotra et al. 2024).

An ethical decision framework

Before any project, work through five questions:

1. **Legality** — CFAA, GDPR, local law, the site's ToS?
2. **Consent** — did people consent? Is it truly public, or shared with a reasonable expectation of limited visibility (Zimmer 2010)?
3. **Proportionality** — only the data you need? Could a smaller or less sensitive set answer the question?
4. **Privacy** — any PII? Could individuals be re-identified? What if it leaks?
5. **Server impact** — a meaningful load? Rate-limited? Scraping off-peak?

No framework substitutes for judgment — but working these questions surfaces the risks.

IRBs & web data

Scraping becomes *human-subjects research* when you collect data about **identifiable people** to produce **generalizable knowledge**.

Public websites: usually not.

Social media profiles: almost certainly.

Wednesday: `robots.txt`

Complete daily note: `https://bit.ly/info4617f26`

Read Textbook Ch. 2 before Wednesday.

Today covered the **law** and the **contract**. Wednesday covers the third layer, the **technical skill**.

You will get the `robots.txt` file of real sites and compare them.

Come with a prediction. Pick two sites you would expect to differ and guess which restricts more, and why. We will check.

Bring a laptop

We send live requests and watch what the server sees. Your `webdata` environment should be working by Wednesday.

Check the handout for tips.

Maybe something is genuinely broken and we need to patch (on Friday)

2. Wednesday · Notebook Lab

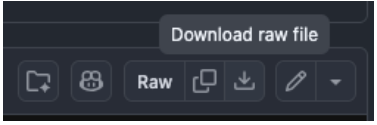
First: get this week's notebook from GitHub

Every week's companion notebook lives in the textbook's GitHub repository, in a `notebooks/` folder. Today's is `ch-02-ethics.ipynb` — this is a habit you'll repeat every Wednesday, so let's do it slowly once:

1. Go to `github.com/cuinfoscience/Web-Data-Science-Book`
2. Open the `notebooks` folder, then click `ch-02-ethics.ipynb`
3. Click **Download raw file** in the file toolbar (the down-arrow icon).
   - If your browser opens the raw JSON instead of downloading it, right-click the button and choose **Save Link As…**
4. Save it into one folder you'll reuse all semester — *e.g.*, `/Documents/INFO4617/` — so you always know where each week's notebook landed

Where to look

The Companion Notebooks appendix of the published book also links every chapter's notebook in one place, if you'd rather click than browse the repo.



Then: launch Jupyter and open the notebook

Jupyter shows you whatever folder it's launched *from* — so `cd` there first, every time:

```
cd /INFO4617 # 1. the folder where you saved the notebook
conda activate webdata # 2. the environment from Week 1's setup.md
jupyter notebook # 3. starts the server; your browser opens automatically
```

In the file-browser tab that opens, click `ch-02-ethics.ipynb` to open it. Before running anything, confirm you're where you think you are:

```
import os
print(os.getcwd())
```

These should print the folder you launched Jupyter from – if it doesn't, stop the server (Ctrl+C), `cd` to the right folder, and relaunch

"Jupyter shows no files"

Almost always the wrong folder. Stop the server, `cd` to where you saved the notebook, relaunch.

"Could not find conda environment: webdata"

Check the `setup.md` handout from week-01.

`robots.txt`: read it before you scrape

A `robots.txt` at a site's root declares what automated agents may fetch. Fetch it **with your own User-Agent**, then parse it and ask permission per URL:

```
import requests
from urllib.robotparser import RobotFileParser

HEADERS = {"User-Agent": "WebDataScience/1.0 (INFO 4617; you@colorado.edu)"}
text = requests.get("https://en.wikipedia.org/robots.txt",
                    headers=HEADERS).text

rp = RobotFileParser()
rp.parse(text.splitlines()) # NOT rp.read() -- see the next slide

rp.can_fetch("*", "https://en.wikipedia.org/wiki/Python_(programming_language)") # True
rp.can_fetch("*", "https://en.wikipedia.org/w/api.php") # False
rp.can_fetch("*", "https://en.wikipedia.org/wiki/Special:Search") # False
```

`/wiki/` articles are open, but `/w/` (dynamic pages + API) and `Special:` are disallowed *for crawlers*. Deliberate API clients follow different norms (Ch. 10).

The problem with `rp.read()`

`RobotFileParser.read()` fetches the file itself, using `urllib` with the default User-Agent `Python-urllib/3.x`. Wikipedia answers that with **403 Forbidden**.

The failure is **silent**. From the standard library:

```
except urllib.error.HTTPError as err:
    if err.code in (401, 403):
        self.disallow_all = True
```

A 403 becomes "this site disallows everything." No exception. Every `can_fetch()` then returns `False`, and you conclude Wikipedia bans crawlers.

This chapter's own lesson

The library that reads the politeness file is impolite by default.

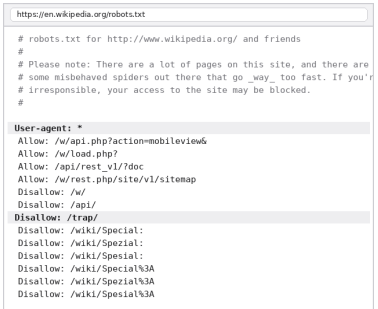
`requests` fails the same way: its default `python-requests/2.x` is also refused.

Fetch with your header, then `rp.parse()`.

What `robots.txt` reveals across platforms

```
sites = {"Wikipedia": "https://en.wikipedia.org/robots.txt",
         "Reddit": "https://www.reddit.com/robots.txt",
         "NY Times": "https://www.nytimes.com/robots.txt"}

for name, url in sites.items():
    lines = requests.get(url, headers=HEADERS).text.strip().split("\n")
    disallow = sum(l.startswith("Disallow") for l in lines)
    allow = sum(l.startswith("Allow") for l in lines)
    print(f"{name}: {len(lines)} lines, {disallow} Disallow, {allow} Allow"
          )
```



Line count is not restrictiveness. Reddit's file is the shortest and the strictest.

Two cautions. First, it is a **norm, not a law**: a violation is not illegal, but it is impolite and the site can block your IP address. Second, a **404** means *no rules*. It does not mean "crawl aggressively."

Activity: survey a sector

Three sites tell you little. A whole sector shows you how a business model affects access. Select one row. Get the `robots.txt` file of each site in that row. Compare them:

- **Search** — `google.com`, `bing.com`, `duckduckgo.com`
- **Social** — `facebook.com`, `instagram.com`, `linkedin.com`, `x.com`, `reddit.com`
- **Commerce** — `amazon.com`, `ebay.com`, `ticketmaster.com`
- **Streaming** — `netflix.com`, `hulu.com`, `spotify.com`
- **News** — `nytimes.com`, `washingtonpost.com`, `wsj.com`, `theguardian.com`
- **Public** — `leg.colorado.gov`, `census.gov`, `sec.gov`

Report back

- Most and least restrictive in your row?
- What does the pattern say about **how each makes money**?
- Does the **public** row look different — and should it?

Which AI crawlers does this site block?

Since 2023, `robots.txt` has grown a second job: naming **AI training crawlers**. Search each file you fetched for:

`GPTBot` (OpenAI) | `ClaudeBot` (Anthropic) | `CCBot` (Common Crawl) | `Google-Extended` | `Applebot-Extended` | `Meta-ExternalAgent` | `PerplexityBot`

Then look for the error that Koebler (2024b) found: a site blocks a **retired** crawler name, but not the operator's **active** one. Copied blocklists go out of date.

`robots.txt` also has no enforcement mechanism. Mehrotra et al. (2024) documents a company that reached blocked pages from an unpublished IP address.

Why respect it

Because your goal is defensible research, not avoiding detection.

The norm does not stop bad actors, but ignoring it will damage your reputation.

Identify yourself, then slow down

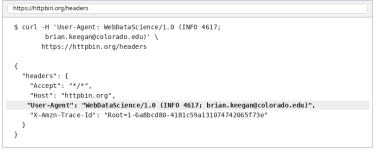
Two habits make you a good citizen of the web — a truthful `User-Agent` and rate limiting:

```python
import time, requests

headers = {
    "User-Agent": "WebDataScience/1.0 (INFO 4617; brian.keegan@colorado.edu
          )"
}

for url in urls:
    response = requests.get(url, headers=headers)
    # ... process the response ...
    time.sleep(1) # a good default: about one request per second
```

The default `requests` User-Agent marks you as an anonymous bot. A good one **identifies you and gives contact info**, so operators can reach you instead of simply blocking you — and some APIs *require* it.



Your header is visible to every server you touch.

`responsible_get()`: build it once

Custom User-Agent, rate limiting, and error handling appear in *every* scraping project. Wrap them into one reusable function instead of re-implementing them ad hoc:

```python
import requests, time

def responsible_get(url, headers=None, delay=1.0):
    """GET with responsible defaults: identify, rate-limit, fail loudly."""
    default_headers = {
        "User-Agent": "WebDataScience/1.0 (INFO 4617; you@colorado.edu)"}
    if headers:
        default_headers.update(headers)
    try:
        response = requests.get(url, headers=default_headers, timeout=30)
        response.raise_for_status() # treat 4xx/5xx as failures, not data
        time.sleep(delay) # rate-limit by default
        return response
    except requests.exceptions.RequestException as e:
        print(f"Request failed ({type(e).__name__}): {url}")
        return None
```

Design choices, and later reuse

Every default encodes a principle

- **Custom User-Agent** — identify & give contact
- `delay` **defaults ON** — aggressive scraping must be a *conscious* override, not an oversight
- `raise_for_status()` — a 403/404 fails loudly instead of feeding your parser garbage
- broad `except` — `ConnectionError`, `Timeout`, `SSLError` log rather than crash the run
- `timeout=30` — never hang forever on a dead server
- pass a site's `Crawl-delay` as `delay` to honor its stated preference

Reused in later chapters

`responsible_get()` returns in **Ch. 6** (scraping HTML tables), **Ch. 7** (Wayback snapshots), and throughout the **API chapters**.

Building responsible defaults into your *tools* is more reliable than remembering to add them each time.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Compare the `robots.txt` files of five sites in one category. Report the patterns that you find.
2. Read one Terms of Service. Find three clauses that limit automated collection. Report if research is an exception.
3. Apply the five-question framework to this plan: collect dating-app profiles to study self-presentation.
4. Measure five requests with `time.time()`. Use a 1-second delay, then a 2-second delay. Compare the times with your intent.
5. Select a site that you use each day. Compare its `robots.txt`, its Terms of Service, and the framework. Write 300 words. Show where they disagree.
6. **5617**: read Fiesler et al. (2020), the EFF page about the CFAA, and *Van Buren*. Then write a two-page memo about collection from platforms

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

3. Friday · Textbook Revisions

Daily note questions, part I

- **Will we use Claude Code, Antigravity, etc.?** — Check out Weeks 13 and 14
- **Why GitHub?** — Common development and project management environment, building a technical portfolio
- **What if you scrape data you shouldn't have?**— Depends on the reason!
- **Final project details?** — Individual submissions but group work mayb e an option
- **Scrape data from a specific API** — Look in `robots.txt` or developer docs for API, we'll discuss hidden APIs and screen scraping later
- **Is INFO1701 enough programming?** — Probably!
- **How are rules enforced?** — Normative, technical, and legal layers for different reasons
- **Changing terms and accessiblity?** — Websites for non-public orgs have very little responsibilty/accountability
- **What to look for in a ToS?** — What data is collected, who can use it, and for what?
- **Making automated scrapers?** — Week 14 or earlier
- **Research institutions scraping?** — All the time but still have to respect rules of the road!
- **How common are civil suits?** — Most likely when using someone else's data for sale or abuse

Daily note questions, part II

- **How much law breaking?** — Often under the broadest reading of CFAA, but rarely-to-never under exceptions for research and teaching exceptions
- **Will scraping ever be illegal?** — Unlikely for technical, legal, and financial reasons
- **Scrape the NYT?** — Use their API! `https://developer.nytimes.com/apis`
- **Scraping for insider trading?** — Not insider anymore once it's on the web, but check out SEC EDGAR `https://www.sec.gov/search-filings/edgar-application-programming-interfaces`
- **Have to use terminal?** — Not necessarily, but it might be worth taking off training wheels. Also check out Missing Manual `https://cuinfoscience.github.io/INFO-Missing-Manual/`
- **Have to run Jupyter in browser?** — Could use Labs or VS Code or similar if you're comfortable with it
- **Do exercises and just submit?** — That's the idea!
- **How to open Terminal? Terminal is scary** — I agree! Keep practicing and check out Missing Manual Ch. 11 `https://cuinfoscience.github.io/INFO-Missing-Manual/`
- **Rate limit on robots.txt?** — Not really

Four objects you'll use all semester

- **Repository** — the project: every file and its full history.
- **Issue** — a written report: "here is a problem." Costs nothing but a browser.
- **Pull request** — a proposed change: "here is the fix, side by side with the original."
- **Review** — a response to a PR: comment line-by-line, then approve or request changes.

Issue vs. pull request

An **issue** says *something is wrong*. A **pull request** says *here is the correction* — and shows the exact diff.

Issues are cheap and fast; PRs are the real contribution.

Today on issues and pull requests later.

Setup: two options

To file an issue you need only a browser.

We will also set up your **tools** now, while I am here to help.

GitHub Desktop — download it, sign in, then Clone the textbook repository. Each later step is a button: branch, commit, push, Create Pull Request.

If you want to be hardcore and use the terminal to interface with GitHub...
The `gh` CLI — install it, run `gh auth login`, then use these commands:

- `gh repo clone cuinfoscience/Web-Data-Science-Book`
- `gh issue create` (yes, you can file today's issue this way)

Checkpoint

You are done when Desktop shows the textbook repository on your computer, or when `gh repo clone` completes with no error.

Revision targets

Goldilocks issues: not too big, not too small.

- **Gaps —** Instructions make an assumption or skip a step that needs to be explained.
- **Bugs —** You ran the code as downloaded and ran into a problem that should just work.
- **New features —** The chapter would be stronger if it covered some other content or exampels.
- **Common Issues to Debug —** Add other issues that that aren't currently included.
- **Add a figure —** Visuals and screenshots are always helpful so identify and suggest things to add.
  - Bonus points for image macro memes that connect with the kids
- **Add a missing citations or references.** The Meta v. Bright Data (2024) and X Corp. rulings are described in prose with no source link.
- **Strengthen an exercise.** Add a rubric or expected output.

File it: your first issue

Work in groups or individually to find, write, and comment on new and existing issues. Issues welcome and credit given on both the Web-Data-Science-book and INFO-Missing-Manual.

1. Find one problem in this week's chapter that no one else has found.
2. If someone else has already beaten you to an issue, add a comment below with any other details or suggestions.
3. Re-run the step from scratch to confirm it's a reproducible problem. Copy the passage, the command, or the output.
4. Go to the textbook GitHub repo. Select the **Issues** tab. Select **New issue**. Complete the skeleton. Select **Submit**.
5. Copy the URL of your issue or comment into the Canvas assignment.

What to submit

You will have two things you need to submit to Canvas every week:

Notebook labs — Wednesday notebook exercises to bottom, save as HTML, submit to Canvas

Textbook revisions — Revisions to textbook and notebook, copy link to your issue or comment, submit to Canvas

Both due Sundays before midnight

Next week: from issues to pull requests

Next week: **The Post-API Age**

- Why the open web is closing
- How researchers are adapting
- What it means for your ability to retrieve data

Notebook lab — Impacts of API limits and auditing API changes

Textbook revisions — Submitting pull requests: if an issue says *something is wrong*, a pull requests say *here is the fix*.

Key takeaways

1. **Great power and responsibility —** Just because you can doesn't mean you should.
2. **The law is unsettled —** new case law revising and rescoping risks under CFAA, but not risk-free
3. `robots.txt` is a **norm, not a law —** read it, respect `Disallow` and `Crawl-delay`.
4. **Identify yourself** (custom `User-Agent`) and **rate-limit** (`time.sleep`) — then wrap both into `responsible_get()` and reuse it.
5. Run the **five-question framework** before every project; when in doubt, use an API, collect less, and err toward caution.